

BDA Lab Exam: Complete Cheat Sheet

Instructions + Sample Input + All Codes

1. General Setup (Run once at start)

```
start-all.sh
jps
mkdir bda_exam && cd bda_exam
```

Experiment 1: Word Count

Sample Input (File: input.txt)

```
hadoop is fun
big data is fun
hadoop is powerful
```

Execution Commands

```
# 1. Create input in HDFS
hdfs dfs -mkdir /input_wc
hdfs dfs -put input.txt /input_wc

# 2. Compile and Run
javac -classpath `hadoop classpath` -d . WordCount*.java
jar -cvf wordcount.jar *.class
hadoop jar wordcount.jar WordCountDriver /input_wc /output_wc

# 3. View Output
hdfs dfs -cat /output_wc/part-r-00000
```

Source Codes

WordCountMapper.java

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class WordCountMapper extends Mapper<Object, Text, Text, IntWritable> {
    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();
    public void map(Object key, Text value, Context context) throws IOException,
        InterruptedException {
        StringTokenizer itr = new StringTokenizer(value.toString());
```

```

        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken());
            context.write(word, one);
        }
    }
}

```

WordCountReducer.java

```

import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class WordCountReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws
        IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) { sum += val.get(); }
        context.write(key, new IntWritable(sum));
    }
}

```

WordCountDriver.java

```

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCountDriver {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Word Count");
        job.setJarByClass(WordCountDriver.class);
        job.setMapperClass(WordCountMapper.class);
        job.setReducerClass(WordCountReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

Experiment 2: Maximum Number

Sample Input (File: numbers.txt)

```
45
12
98
34
```

Execution Commands

```
hdfs dfs -mkdir /input_max
hdfs dfs -put numbers.txt /input_max
javac -classpath `hadoop classpath` -d . Max*.java
jar -cvf max.jar *.class
hadoop jar max.jar MaxDriver /input_max /output_max
hdfs dfs -cat /output_max/part-r-00000
```

Source Codes

MaxMapper.java

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class MaxMapper extends Mapper<Object, Text, Text, IntWritable> {
    private Text word = new Text("max");
    public void map(Object key, Text value, Context context) throws IOException,
        InterruptedException {
        int number = Integer.parseInt(value.toString());
        context.write(word, new IntWritable(number));
    }
}
```

MaxReducer.java

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class MaxReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws
        IOException, InterruptedException {
        int max = Integer.MIN_VALUE;
        for (IntWritable val : values) {
            if (val.get() > max) { max = val.get(); }
        }
        context.write(key, new IntWritable(max));
    }
}
```

MaxDriver.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MaxDriver {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Maximum Number");
        job.setJarByClass(MaxDriver.class);
        job.setMapperClass(MaxMapper.class);
        job.setReducerClass(MaxReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Experiment 3: Filter Data (Map-Only)

Sample Input (File: employees.txt)

```
101,Amit,HR
102,Sita,IT
103,Rahul,HR
104,Vijay,Sales
```

Execution Commands

```
hdfs dfs -mkdir /input_filter
hdfs dfs -put employees.txt /input_filter
javac -classpath `hadoop classpath` -d . Filter*.java
jar -cvf filter.jar *.class
hadoop jar filter.jar FilterDriver /input_filter /output_filter
hdfs dfs -cat /output_filter/part-m-000000
```

Source Codes

FilterMapper.java

```
import java.io.IOException;
import org.apache.hadoop.io.NullWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class FilterMapper extends Mapper<Object, Text, NullWritable, Text> {
    public void map(Object key, Text value, Context context) throws IOException,
        InterruptedException {
        String[] fields = value.toString().split(",");
        if (fields[2].equals("HR")) {
            context.write(NullWritable.get(), value);
        }
    }
}
```

FilterDriver.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.NullWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class FilterDriver {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Filter Data");
        job.setJarByClass(FilterDriver.class);
        job.setMapperClass(FilterMapper.class);
        job.setNumReduceTasks(0);
        job.setOutputKeyClass(NullWritable.class);
```

```
        job.setOutputValueClass(Text.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Experiment 4: Average Calculation

Sample Input (File: scores.txt)

```
10.0
20.0
30.0
```

Execution Commands

```
hdfs dfs -mkdir /input_avg
hdfs dfs -put scores.txt /input_avg
javac -classpath `hadoop classpath` -d . Avg*.java
jar -cvf avg.jar *.class
hadoop jar avg.jar AvgDriver /input_avg /output_avg
hdfs dfs -cat /output_avg/part-r-00000
```

Source Codes

AvgMapper.java

```
import java.io.IOException;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class AvgMapper extends Mapper<Object, Text, Text, DoubleWritable> {
    private Text word = new Text("avg");
    public void map(Object key, Text value, Context context) throws IOException,
        InterruptedException {
        double num = Double.parseDouble(value.toString());
        context.write(word, new DoubleWritable(num));
    }
}
```

AvgReducer.java

```
import java.io.IOException;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class AvgReducer extends Reducer<Text, DoubleWritable, Text, DoubleWritable> {
    public void reduce(Text key, Iterable<DoubleWritable> values, Context context) throws
        IOException, InterruptedException {
        double sum = 0; int count = 0;
        for (DoubleWritable val : values) { sum += val.get(); count++; }
        context.write(key, new DoubleWritable(sum / count));
    }
}
```

AvgDriver.java

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class AvgDriver {
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Average Calculation");
        job.setJarByClass(AvgDriver.class);
        job.setMapperClass(AvgMapper.class);
        job.setReducerClass(AvgReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(DoubleWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Important Troubleshooting

CRITICAL: If you need to re-run any experiment, delete the output folder first or Hadoop will crash:

```
hdfs dfs -rm -r /output_name
```